

PROJECT REPORT

Design and Simulation of an Automatic Water Level Indicator Using Arduino and Ultrasonic Sensor

An embedded systems project implemented and verified on the Tinkercad circuit simulation platform

Detail	Information
Project Title	Automatic Water Level Indicator Using Arduino and Ultrasonic Sensor
Student Name	Kiran , Hoorulain and Mehak_____
Roll Number / ID	124-BSCS/25-717 124-BSCS/25-747 124-BSCS/25-724_____
Department / Class	_BSCS 2 nd Semester_____
Institution Name	National University of Pakistan_____
Supervisor / Instructor	_Sir Huzaifa_____
Submission Date	__6/25/2026_____

Academic Session 2025 - 2026

Table of Contents

1. Abstract.....	4
2. Introduction	4
2.1 Background of Water Level Monitoring	4
2.2 Importance of Automatic Water Level Indication	4
2.3 Real-World Applications	4
3. Problem Statement	5
4. Objectives	5
5. Literature Review	6
5.1 Existing Water Level Monitoring Methods	6
5.2 Arduino-Based Monitoring Systems.....	6
5.3 Ultrasonic Sensing Technology	6
6. Methodology	6
6.1 Working Principle.....	6
6.2 Distance Calculation Using Ultrasonic Waves	6
6.3 Water Level Percentage Calculation.....	7
6.4 Alert and Control Mechanism.....	7
7. Components Required	8
8. Circuit Design and Connections	8
8.1 Ultrasonic Sensor (HC-SR04) Pins.....	8
8.2 LCD 16x2 Interfacing (4-bit Mode).....	9
8.3 Actuator and Power Distribution	9
8.4 Simulation Circuit Diagram	9
9. Software Design.....	10
9.1 Development Environment - Arduino IDE	10
9.2 Program Logic	10
9.3 Flowchart.....	11
9.4 Embedded Code Explanation	11
10. Simulation Setup (Tinkercad)	12
10.1 Simulation Environment.....	12
10.2 Testing Procedure	12
10.3 Screenshots	Error! Bookmark not defined.
11. Results and Discussion	13
11.1 Sensor Readings and LCD Output.....	13
11.2 Alarm / Actuator Operation	13
11.3 Accuracy Analysis	13

12. Advantages	13
13. Limitations	14
14. Applications	14
15. Future Improvements	14
16. Conclusion	15
17. References	15
18. Appendix	15
18.1 Appendix A - Arduino Code	15
18.2 Appendix B - Circuit Diagram	16

1. Abstract

This report presents the design, simulation, and implementation of an Automatic Water Level Indicator System built around the Arduino Uno microcontroller and the HC-SR04 ultrasonic distance sensor. The system continuously measures the distance between a fixed reference point at the top of a storage tank and the surface of the water, converts that distance into a water level expressed as a percentage of full capacity, and presents the value in real time on a 16x2 alphanumeric LCD. When the water reaches a predefined threshold the controller switches an actuator connected to a PWM-capable pin, which in the simulation drives a fill pump (and may equivalently sound a buzzer as an overflow alarm). The complete circuit was assembled and verified inside the Tinkercad simulation environment, removing the need for physical hardware during development and testing.

Objective: The objective of the project is to replace error-prone manual inspection of water storage tanks with an automatic, contactless, and continuously updating electronic indicator that both displays the level and controls the filling action at safe limits.

Expected Outcomes: A working simulated system that (a) reports water level as a 0-100% figure on an LCD, (b) updates continuously as the level changes, and (c) automatically starts the fill actuator when the tank falls below the lower threshold and stops it as the tank approaches full, thereby preventing both dry-running and overflow.

2. Introduction

2.1 Background of Water Level Monitoring

Water is stored in overhead and underground tanks in almost every home, school, factory, and farm. Knowing how much water remains in a tank, and being warned before it overflows or runs dry, is a basic but recurring engineering problem. Traditional approaches range from simply climbing up and looking into the tank, to mechanical float gauges, to electrode-based conductive sensors. Each of these has practical drawbacks: visual checking is slow and unsafe, float mechanisms wear out and stick, and conductive probes corrode and contaminate the water over time.

2.2 Importance of Automatic Water Level Indication

An automatic indicator removes the human from the routine monitoring loop. Instead of remembering to check the tank, the user simply reads a display or relies on the controller to start and stop a pump automatically. This conserves water, saves electricity by preventing pumps from running dry, avoids the structural damage and waste caused by overflow, and provides a continuous, objective measurement rather than an occasional guess.

2.3 Real-World Applications

- Domestic overhead and underground water tanks in homes and apartment buildings.
- Automatic pump control systems that switch a motor on and off based on level.
- Industrial process tanks, chemical reservoirs, and coolant storage.

- Agricultural irrigation reservoirs and livestock water troughs.
- Municipal water-supply and overhead distribution tanks.

3. Problem Statement

Manual monitoring of water tanks is inefficient and unreliable. A person has to physically inspect the tank, which is inconvenient for elevated or enclosed tanks and impossible to do continuously. As a result, tanks frequently overflow, wasting treated water and damaging walls and ceilings, or run completely empty, leaving the pump to run dry and risking motor burnout.

The specific shortcomings of manual tank monitoring are:

- It requires constant human attention, which is impractical around the clock.
- Overflow is detected only after water is already being wasted.
- Dry-running of the pump is detected only after the motor is already stressed.
- It provides no recorded or numeric value, only a rough visual estimate.

There is therefore a clear need for automation: a low-cost electronic system that senses the level without contact, displays it numerically, and acts on predefined thresholds without any human intervention.

4. Objectives

1. To measure the water level inside a tank automatically and without physical contact with the water, using ultrasonic distance sensing.
2. To display the real-time level as an easily understood percentage (0-100%) on a 16x2 LCD screen.
3. To generate an automatic response at predefined thresholds, namely starting the fill actuator when the tank is low and stopping it as the tank approaches full to prevent overflow.
4. To design and verify the entire system in a simulation environment (Tinkercad) before any physical hardware is purchased.

5. Literature Review

5.1 Existing Water Level Monitoring Methods

Several sensing technologies have historically been used for liquid level measurement. Float-based gauges use a buoyant body coupled to a mechanical or magnetic indicator; they are simple but suffer from mechanical wear and sticking. Conductive (electrode) sensors detect the level by completing a circuit through the water at fixed heights; they are cheap but corrode and are limited to discrete steps rather than a continuous reading. Capacitive and pressure (hydrostatic) sensors offer continuous output but are comparatively expensive and require calibration.

5.2 Arduino-Based Monitoring Systems

The Arduino platform has become a popular foundation for low-cost instrumentation because it combines an inexpensive microcontroller with a large library ecosystem and a simple programming model. Numerous published student and hobbyist projects pair an Arduino with a sensor and a display or a relay to build level monitors, automatic pump controllers, and alarm systems. The Arduino handles the timing-sensitive measurement, the arithmetic that converts a raw reading into a meaningful quantity, and the decision logic that drives outputs, all on a single board.

5.3 Ultrasonic Sensing Technology

Ultrasonic ranging measures distance by emitting a short burst of high-frequency sound (typically 40 kHz for the HC-SR04) and timing how long the echo takes to return after reflecting from a surface. Because it is contactless, an ultrasonic sensor never touches the liquid, so it does not corrode, does not contaminate the water, and works with almost any liquid regardless of conductivity. Mounted at the top of a tank facing downward, the sensor reports the air gap above the water; as the tank fills, that gap shrinks, giving an inverse relationship between measured distance and water level that the controller converts into a percentage.

6. Methodology

6.1 Working Principle

The HC-SR04 is mounted at the top of the tank, pointing down at the water surface. On each measurement cycle the Arduino sends a short pulse to the sensor's TRIG pin. The sensor transmits an ultrasonic burst and then raises its ECHO pin for a duration proportional to the round-trip travel time of the sound. The Arduino measures this ECHO pulse width, converts it to a distance, and from the distance computes how full the tank is.

6.2 Distance Calculation Using Ultrasonic Waves

Sound travels through air at approximately 343 metres per second, which is about 0.0343 cm per microsecond. Because the pulse must travel to the water surface and back, the one-way

distance is half of the total path. The firmware reads the echo time in microseconds with `pulseIn()` and multiplies it by the constant 0.01723, which already folds in both the speed of sound and the division by two:

$$\text{distance (cm)} = 0.01723 \times \text{echo_time (microseconds)}$$

For example, an echo time of 5000 microseconds corresponds to about 86 cm of air gap above the water surface.

6.3 Water Level Percentage Calculation

The tank in this design is sized for a 600 litre capacity, for which the empty-tank air gap (sensor to tank floor) is taken as 334 cm. The measured distance is mapped onto a 0-100% scale using the Arduino `map()` function. When the air gap equals 334 cm the tank is empty and the level reads 0%; when the gap approaches 0 cm the water is near the sensor and the level reads 100%:

$$\text{level (\%)} = \text{map}(\text{distance}, 334, 0, 0, 100)$$

Because the input range (334 -> 0) is reversed relative to the output range (0 -> 100), the mapping correctly produces a high percentage for a small air gap and a low percentage for a large air gap.

6.4 Alert and Control Mechanism

The computed level drives an actuator on PWM pin 11 through `analogWrite()`. The control logic is threshold based. While the level is below 95% the actuator is energised; this represents a fill pump turning on to top up the tank (and equivalently is where a buzzer would sound a low-level / filling alert). Once the level reaches 99% or above, the actuator is switched off so that filling stops before the tank can overflow. The intermediate 95-98% band is deliberately held off as a safety hysteresis margin so the output does not chatter rapidly on and off near the full mark.

7. Components Required

#	Component	Function in the System
1	Arduino Uno (ATmega328P)	Main controller; triggers the sensor, measures the echo, computes the level, drives the LCD and the actuator.
2	HC-SR04 Ultrasonic Sensor	Contactless distance measurement between the sensor and the water surface.
3	LCD 16x2 Display	Shows the heading and the real-time water level percentage.
4	Potentiometer (10k)	Adjusts the LCD contrast (connected to the LCD V0 / contrast pin).
5	Buzzer / DC Motor (actuator)	Output device on pin 11; sounds the alarm or drives the fill pump at the set thresholds.
6	Breadboard	Solderless platform for distributing power and interconnecting components.
7	Connecting / Jumper Wires	Carry signals and power between the Arduino, breadboard, sensor, LCD, and actuator.
8	Resistors	Current limiting and pull-down where required (e.g., for the actuator / signal lines).
9	Power Supply (USB / 9V)	Provides 5V regulated power to the Arduino and the connected components.

8. Circuit Design and Connections

All components share a common 5V and ground distributed along the breadboard power rails from the Arduino's 5V and GND pins. The wiring follows directly from the pin assignments used in the firmware.

8.1 Ultrasonic Sensor (HC-SR04) Pins

Sensor Pin	Connects To	Purpose
VCC	5V rail	Power supply to the sensor
GND	Ground rail	Common ground
TRIG	Arduino pin 9	Receives the trigger pulse from

Sensor Pin	Connects To	Purpose
		the Arduino
ECHO	Arduino pin 10	Returns the echo pulse whose width encodes distance

8.2 LCD 16x2 Interfacing (4-bit Mode)

The LCD is driven in 4-bit parallel mode, which is exactly what the firmware declares with `LiquidCrystal lcd(2,3,4,5,6,7)`. Only four data lines plus two control lines are used, freeing the remaining Arduino pins.

LCD Pin	Connects To	Role
RS	Arduino pin 2	Register Select
E (Enable)	Arduino pin 3	Enable strobe
D4	Arduino pin 4	Data bit 4
D5	Arduino pin 5	Data bit 5
D6	Arduino pin 6	Data bit 6
D7	Arduino pin 7	Data bit 7
VSS / RW / K	Ground rail	Logic ground and backlight cathode
VDD / A	5V rail	Logic power and backlight anode
V0 (contrast)	Potentiometer wiper	Sets display contrast

8.3 Actuator and Power Distribution

- The actuator (buzzer or DC motor representing the pump) is driven from Arduino pin 11, which supports `analogWrite()` / PWM.
- The potentiometer's two outer legs go to 5V and GND, and its wiper feeds the LCD contrast pin, allowing the text to be made clearly visible.
- The Arduino is powered through its USB connector (or a 9V supply on the barrel jack), and its on-board regulator supplies the 5V rail that feeds every other component.

8.4 Simulation Circuit Diagram

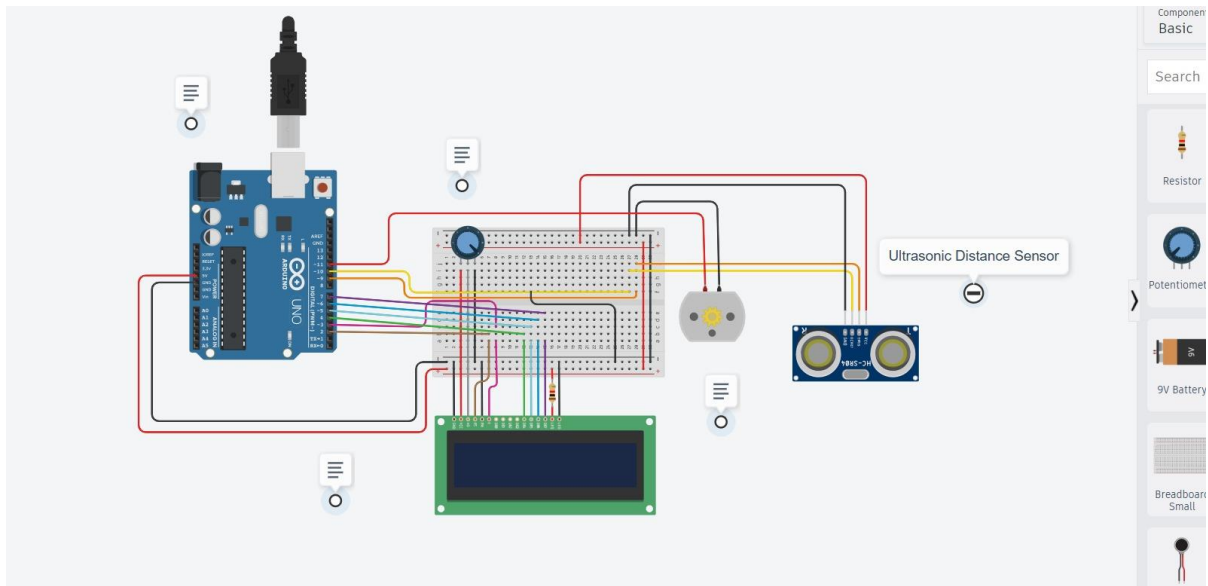


Figure 1: Tinkercad circuit of the Automatic Water Level Indicator (Arduino Uno, HC-SR04, 16x2 LCD, potentiometer, actuator).

9. Software Design

9.1 Development Environment - Arduino IDE

The firmware was written in C/C++ using the Arduino IDE. The IDE provides the compiler, the LiquidCrystal library used to drive the display, and the upload toolchain. Within Tinkercad the same code editor and the standard Arduino libraries are available, so the program runs identically in simulation and on real hardware.

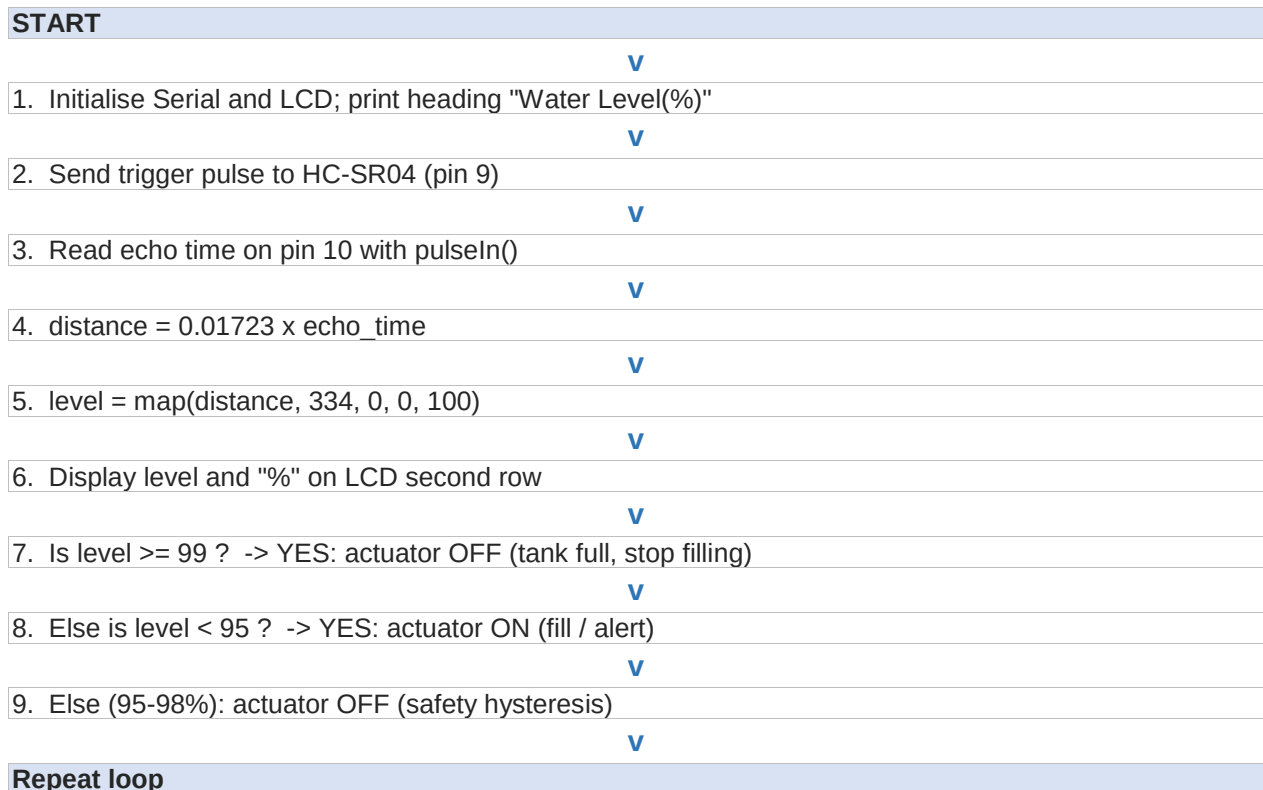
9.2 Program Logic

1. In `setup()`, the serial port and the LCD are initialised, the screen is cleared, and the fixed heading "Water Level(%)" is printed on the top row.
2. In `loop()`, the program triggers the ultrasonic sensor, reads the echo time, and converts it to a distance in centimetres.

3. The distance is mapped to a 0-100% level and printed on the second row of the LCD, followed by a percent sign.
4. The level is compared against the thresholds, and pin 11 is switched on or off accordingly to control the actuator.
5. The loop repeats continuously, so the displayed value and the actuator state track the water level in real time.

9.3 Flowchart

The overall flow of the firmware is summarised below:



9.4 Embedded Code Explanation

The key parts of the program operate as follows:

- **Library and object:** `#include<LiquidCrystal.h>` brings in the display driver, and `LiquidCrystal lcd(2,3,4,5,6,7)` creates the LCD object using RS=2, E=3 and data pins D4-D7 on pins 4-7 (4-bit mode).
- **UltrasonicDistance():** This function configures the trigger as an output and the echo as an input, emits the trigger pulse, and returns the echo pulse width measured by `pulseIn()`. That returned time is the raw input for the distance calculation.
- **Distance conversion:** `float dist = 0.01723 * UltrasonicDistance(9,10);` turns the echo time into a one-way distance in centimetres.
- **Level mapping:** `int level = map(dist, 334, 0, 0, 100);` scales the distance to a percentage, with 334 cm representing the empty 600 L tank.

- **Display:** `lcd.setCursor()` and `lcd.print()` write the numeric level and the % symbol onto the second row, beneath the fixed heading.
- **Threshold control:** The if / else-if / else block drives pin 11 with `analogWrite()`: off at $\geq 99\%$, on at $< 95\%$, and off in the 95-98% hysteresis band.

Note (engineering observations for transparency):

- In `UltrasonicDistance()`, `digitalWrite(trigPin, HIGH)` is issued twice; conventionally the trigger line should be set LOW first to guarantee a clean rising edge. This does not stop the simulation from running but is worth correcting on hardware.
- `map()` takes integer arguments, so the float distance is truncated to a whole number before mapping; this is acceptable here because a 1 cm resolution is more than adequate for tank monitoring.
- When the level reaches 100 (three digits), the "%" printed at column 3 can sit immediately next to the number; shifting the cursor or clearing the row would tidy this edge case.

10. Simulation Setup (Tinkercad)

10.1 Simulation Environment

Tinkercad Circuits is a free, browser-based electronics simulator that includes an Arduino Uno model, an HC-SR04 sensor, a 16x2 LCD, breadboards, potentiometers, and output devices, together with a built-in Arduino code editor and serial monitor. It allows a complete circuit to be assembled, programmed, and run virtually, so the design can be validated before any physical components are bought.

10.2 Testing Procedure

1. Place the Arduino Uno, breadboard, HC-SR04, 16x2 LCD, potentiometer, and actuator on the workspace, then wire them according to Section 8.
2. Open the code editor, paste the firmware, and confirm the LCD declaration and sensor pins match the wiring.
3. Start the simulation and adjust the potentiometer until the LCD text is clearly legible.
4. Vary the distance reported by the ultrasonic sensor (using the sensor's distance slider in Tinkercad) to emulate the water rising and falling.
5. Observe the level percentage on the LCD and confirm the actuator switches on below 95% and off at and above 99%.

11. Results and Discussion

11.1 Sensor Readings and LCD Output

Across the tested range the system behaved as designed. As the simulated air gap was reduced (water rising), the computed distance fell and the level percentage rose, and vice versa. Representative readings are tabulated below for the 600 L tank with a 334 cm empty-tank reference.

Air Gap (cm)	Computed Level (%)	LCD Display	Actuator State
334	0	0 %	ON (filling)
250	25	25 %	ON (filling)
167	50	50 %	ON (filling)
84	75	75 %	ON (filling)
20	94	94 %	ON (filling)
17	95	95 %	OFF (hysteresis)
3	99	99 %	OFF (full)
0	100	100 %	OFF (full)

11.2 Alarm / Actuator Operation

The threshold logic produced the intended behaviour: the actuator on pin 11 remained energised while the level was below 95%, switched off as the level entered the 95-98% safety band, and stayed off at 99% and above. This prevents overflow at the top of the tank and, by the same logic, provides the basis for an audible overflow alarm if a buzzer is used in place of the pump motor.

11.3 Accuracy Analysis

In the ideal simulation environment the measured distance tracks the set value almost exactly, so the percentage is accurate to within the 1 cm rounding introduced by integer mapping. On real hardware, accuracy would additionally depend on the speed of sound varying with air temperature, the beam reflecting cleanly from a flat water surface, and the sensor's rated range (the HC-SR04 is specified for roughly 2 cm to 400 cm). For tanks deeper than the sensor's range, the geometry or the reference distance would need to be adapted. Overall, the simulation confirms that the design meets its objectives of contactless measurement, real-time percentage display, and automatic threshold control.

12. Advantages

- Contactless measurement: the sensor never touches the water, so there is no corrosion or contamination.

- Continuous, numeric output: the level is shown as a clear 0-100% value rather than a rough estimate.
- Automatic control: the pump/alarm is driven automatically at safe thresholds without human attention.
- Low cost and low power: built from inexpensive, widely available components driven by a single Arduino.
- Simulation-first: the whole system is validated in Tinkercad before spending money on hardware.
- Easily extendable: thresholds, tank size, and outputs can be changed by editing a few constants.

13. Limitations

- Range bound: the HC-SR04 works reliably only up to about 4 metres, limiting maximum tank depth.
- Surface dependence: foam, turbulence, or a steeply angled water surface can scatter the echo and disturb the reading.
- Environmental sensitivity: on real hardware the speed of sound shifts with temperature and humidity, introducing small errors.
- Single reference value: the empty-tank distance (334 cm) is hard-coded, so a different tank requires re-calibration in the code.
- No data logging or remote alerts: the current design only displays locally and switches one output; it does not store history or notify remotely.

14. Applications

- Automatic overhead tank monitoring and pump control in homes and apartments.
- Water management in schools, hospitals, hostels, and offices.
- Industrial storage tanks for water, fuel, or non-corrosive chemicals.
- Agricultural irrigation tanks and livestock water supplies.
- Rainwater harvesting reservoirs where overflow must be prevented.

15. Future Improvements

- Add a dedicated buzzer alongside the pump so that overflow and low-level conditions are both indicated audibly.

- Add temperature compensation so the speed-of-sound constant adjusts automatically for more accurate distance.
- Add Wi-Fi or GSM (e.g., using an ESP8266/ESP32) for remote monitoring and mobile notifications.
- Log readings to an SD card or cloud dashboard to track usage trends over time.
- Replace hard-coded thresholds with adjustable settings, for example through extra potentiometers or push-buttons and a menu.
- Use a relay or motor driver so the same logic can switch a real mains pump safely.

16. Conclusion

This project successfully designed, simulated, and implemented an Automatic Water Level Indicator using an Arduino Uno and an HC-SR04 ultrasonic sensor. By measuring the air gap above the water without any physical contact, converting it into a percentage, and displaying it live on a 16x2 LCD, the system replaces unreliable manual inspection with continuous, objective monitoring. The threshold-based control on pin 11 automatically manages the fill actuator and provides the basis for overflow alerting, preventing both dry-running and overflow. The entire design was verified in Tinkercad, demonstrating that a practical, low-cost water management solution can be developed and validated entirely in simulation. The project meets all of its stated objectives and offers a clear, extensible foundation for more advanced features such as remote monitoring and data logging.

17. References

- [1] Arduino. Arduino Uno Rev3 - Technical Specifications and Documentation. Arduino Official Website.
- [2] Arduino. LiquidCrystal Library Reference. Arduino Official Documentation.
- [3] Cytron / Elec Freaks. HC-SR04 Ultrasonic Ranging Module Datasheet.
- [4] Autodesk. Tinkercad Circuits - Getting Started with Arduino Simulation. Autodesk, Inc.
- [5] Hitachi. HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller / Driver Datasheet.
- [6] Margolis, M. Arduino Cookbook. O'Reilly Media.
- [7] Banzi, M., & Shiloh, M. Getting Started with Arduino. Maker Media.

18. Appendix

18.1 Appendix A - Arduino Code

The complete firmware uploaded to the Arduino Uno is reproduced below exactly as used in the simulation.

```
//for 600 Litre Water Tank
/*#define trigPin 9;
#define echoPin 10;*/
long duration;
#include<LiquidCrystal.h>
LiquidCrystal lcd(2,3,4,5,6,7);
long UltrasonicDistance(int trigPin, int echoPin)
{
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    //float dist;
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(2);
    digitalWrite(trigPin,HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    return pulseIn(echoPin, HIGH);
}
void setup()
{
    Serial.begin(9600);
    lcd.begin(16,2);
    lcd.clear();
    lcd.print("Water Level(%)");
}
void loop()
{
    float dist = 0.01723 * UltrasonicDistance(9,10);
    int level = map(dist, 334 , 0, 0 , 100);
    lcd.setCursor(0,1);
    lcd.print(level);
    lcd.setCursor(3,1);
    lcd.print("%");
    int speed=10;

    if( level >=99)
    {
        analogWrite(11,0);
    }
    else if( level <95)
    {
        analogWrite(11,speed);
    }

    else
    {
        analogWrite(11,0); // this is for safety in case.
    }
}
```

18.2 Appendix B - Circuit Diagram

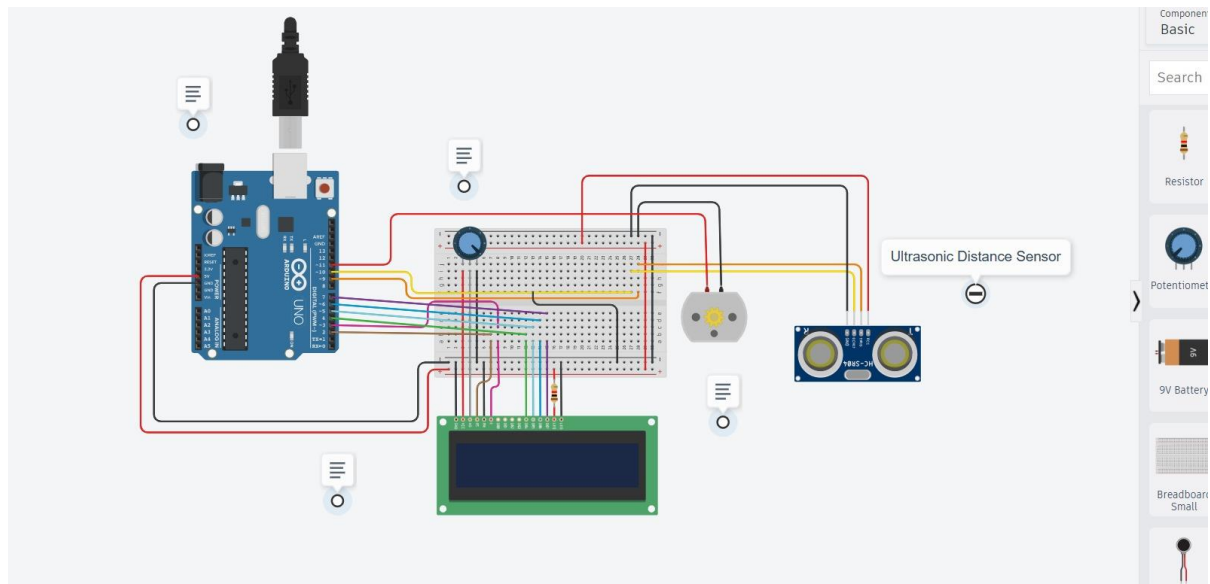


Figure 2: Complete Tinkercad wiring diagram for the Automatic Water Level Indicator.